```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width,initial-scale=1.0"/>
<title>LibraryOS â€” Dark Glassmorphism Library Management System</title>
<!-- Google Fonts: Inter & Sora -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Sora:wght@400;500;600;700;800&display=swap" rel="stylesheet">
<!-- Tailwind CSS CDN -->
<script src="https://cdn.tailwindcss.com"></script>
<!-- React 18 & Babel CDN -->
<script src="https://unpkg.com/react@18/umd/react.production.min.js"></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.production.min.js"></script>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

<style>
  :root {
    --bg-base: #0B0B1A;
    --orb-1: #3D1A78;
    --orb-2: #1A237E;
    --orb-3: #4A148C;
    --orb-4: #0D47A1;
    --orb-5: #1B0036;
    --glass-bg: rgba(255, 255, 255, 0.05);
    --glass-border: rgba(255, 255, 255, 0.12);
    --glass-hover: rgba(255, 255, 255, 0.09);
    --purple-light: #CE93D8;
    --purple-main: #9C27B0;
    --purple-bright: #E040FB;
    --blue-light: #90CAF9;
    --blue-main: #1565C0;
    --blue-bright: #448AFF;
    --text-primary: #F3E5F5;
    --text-secondary: #CE93D8;
    --text-muted: #7B6F8A;
    --success: #69F0AE;
    --danger: #FF5252;
    --warning: #FFD740;
    --accent: #EA80FC;
  }

  * { box-sizing: border-box; margin: 0; padding: 0; }
  html, body, #root {
    height: 100%; width: 100%; overflow: hidden;
    background-color: var(--bg-base); color: var(--text-primary);
    font-family: 'Inter', sans-serif;
  }
  h1, h2, h3, h4, .font-heading { font-family: 'Sora', sans-serif; }

  /* Custom Scrollbar */
  ::-webkit-scrollbar { width: 6px; height: 6px; }
  ::-webkit-scrollbar-track { background: transparent; }
  ::-webkit-scrollbar-thumb { background: rgba(156, 39, 176, 0.5); border-radius: 9999px; }
  ::-webkit-scrollbar-thumb:hover { background: rgba(224, 64, 251, 0.7); }

  /* Canvas & Mesh Grid */
  #orbCanvas { position: fixed; inset: 0; width: 100%; height: 100%; z-index: 0; pointer-events: none; }
  .mesh-grid {
    position: fixed; inset: 0; width: 100%; height: 100%; z-index: 1; pointer-events: none;
    background-image:
      linear-gradient(to right, rgba(255, 255, 255, 0.05) 1px, transparent 1px),
      linear-gradient(to bottom, rgba(255, 255, 255, 0.05) 1px, transparent 1px);
    background-size: 50px 50px;
    animation: gridMove 40s linear infinite;
  }
  @keyframes gridMove { 0% { background-position: 0 0; } 100% { background-position: 50px 50px; } }

  /* Page progress bar */
  .page-progress-bar {
    position: fixed; top: 0; left: 0; height: 3px;
    background: linear-gradient(90deg, #9C27B0, #E040FB, #448AFF);
    z-index: 99999; box-shadow: 0 0 10px #E040FB;
    transition: width 0.3s ease, opacity 0.4s ease;
  }

  /* Glass Panels */
  .glass-card {
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.12);
    border-radius: 20px;
    backdrop-filter: blur(20px) saturate(180%);
    -webkit-backdrop-filter: blur(20px) saturate(180%);
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);
    transition: transform 0.3s ease, box-shadow 0.3s ease, border-color 0.3s ease;
  }
  .glass-card-hover:hover {
    transform: translateY(-4px);
    box-shadow: 0 14px 40px rgba(0, 0, 0, 0.5), inset 0 0 20px rgba(224, 64, 251, 0.06);
    border-color: rgba(224, 64, 251, 0.3);
  }
  .glass-header {
    background: rgba(255, 255, 255, 0.04); border-bottom: 1px solid rgba(255, 255, 255, 0.08);
    backdrop-filter: blur(30px); -webkit-backdrop-filter: blur(30px);
  }
  .glass-sidebar {
    background: rgba(255, 255, 255, 0.04); border-right: 1px solid rgba(255, 255, 255, 0.08);
    backdrop-filter: blur(30px); -webkit-backdrop-filter: blur(30px);
  }
  .glass-modal {
    background: rgba(15, 10, 30, 0.9); border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 24px; backdrop-filter: blur(40px); -webkit-backdrop-filter: blur(40px);
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.7), 0 0 40px rgba(156, 39, 176, 0.25);
    animation: modalIn 0.25s cubic-bezier(0.16, 1, 0.3, 1) forwards;
  }
  @keyframes modalIn {
    from { opacity: 0; transform: scale(0.95); }
    to { opacity: 1; transform: scale(1); }
  }

  /* Buttons */
  .btn-primary {
    background: linear-gradient(135deg, #9C27B0, #1565C0);
    color: #fff; font-weight: 600; border-radius: 12px; border: none; cursor: pointer;
    transition: all 0.25s ease; box-shadow: 0 4px 15px rgba(156, 39, 176, 0.3);
  }
  .btn-primary:hover {
    background: linear-gradient(135deg, #B715C8, #1976D2);
    box-shadow: 0 6px 25px rgba(224, 64, 251, 0.45); transform: scale(1.02);
  }
  .btn-primary:active { transform: scale(0.97); }

  .btn-danger {
    background: linear-gradient(135deg, #FF5252, #B71C1C);
    color: #fff; font-weight: 600; border-radius: 12px; border: none; cursor: pointer;
    transition: all 0.25s ease; box-shadow: 0 4px 15px rgba(255, 82, 82, 0.3);
  }
  .btn-danger:hover { box-shadow: 0 6px 20px rgba(255, 82, 82, 0.5); transform: scale(1.02); }
  .btn-danger:active { transform: scale(0.97); }

  .btn-secondary {
    background: rgba(255, 255, 255, 0.08); border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 12px; color: #CE93D8; font-weight: 500; cursor: pointer; transition: all 0.25s ease;
  }
  .btn-secondary:hover {
    background: rgba(255, 255, 255, 0.14); color: #fff; border-color: rgba(224, 64, 251, 0.4); transform: scale(1.02);
  }
  .btn-secondary:active { transform: scale(0.97); }

  .btn-glass-pill {
    background: rgba(255, 255, 255, 0.08); border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 9999px; color: var(--text-primary); cursor: pointer; transition: all 0.25s ease;
  }
  .btn-glass-pill:hover {
    background: rgba(224, 64, 251, 0.2); border-color: #E040FB; box-shadow: 0 0 15px rgba(224, 64, 251, 0.35);
  }

  /* Form Inputs & Floating Labels */
  .floating-input-wrap { position: relative; margin-bottom: 1.25rem; }
  .floating-input {
    width: 100%; background: rgba(255, 255, 255, 0.06); border: 1px solid rgba(255, 255, 255, 0.12);
    border-radius: 12px; color: #ffffff; padding: 1rem 1rem 0.5rem 1rem; font-size: 0.95rem; outline: none;
    transition: all 0.25s ease;
  }
  .floating-input:focus {
    border-color: #E040FB; box-shadow: 0 0 16px rgba(224, 64, 251, 0.35); background: rgba(255, 255, 255, 0.09);
  }
  .floating-label {
    position: absolute; left: 1rem; top: 0.75rem; color: var(--text-muted); font-size: 0.85rem;
    pointer-events: none; transition: all 0.2s ease; transform-origin: left top;
  }
  .floating-input:focus ~ .floating-label,
  .floating-input:not(:placeholder-shown) ~ .floating-label {
    top: 0.25rem; font-size: 0.7rem; color: #CE93D8; font-weight: 600;
  }
  .input-valid-check {
    position: absolute; right: 1rem; top: 0.85rem; color: #69F0AE; font-size: 1rem; pointer-events: none;
  }

  select.glass-select {
    width: 100%;
    background: rgba(255, 255, 255, 0.06) url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' fill='%23CE93D8' viewBox='0 0 24 24'%3E%3Cpath d='M7 10l5 5 5-5z'/%3E%3C/svg%3E") no-repeat right 0.75rem center;
    background-size: 1.5rem; border: 1px solid rgba(255, 255, 255, 0.12); border-radius: 12px; color: #ffffff;
    padding: 0.75rem 2.2rem 0.75rem 1rem; font-size: 0.95rem; outline: none; appearance: none;
    transition: all 0.25s ease;
  }
  select.glass-select:focus { border-color: #E040FB; box-shadow: 0 0 16px rgba(224, 64, 251, 0.35); }
  select.glass-select option { background: #150E26; color: #F3E5F5; }

  /* Table Styles */
  table { width: 100%; border-collapse: separate; border-spacing: 0; }
  thead th {
    background: rgba(255, 255, 255, 0.08); color: #E040FB; font-family: 'Sora', sans-serif;
    font-size: 0.72rem; font-weight: 700; letter-spacing: 0.06em; text-transform: uppercase;
    padding: 13px 16px; text-align: left; user-select: none; cursor: pointer;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
  }
  thead th:first-child { border-top-left-radius: 12px; }
  thead th:last-child { border-top-right-radius: 12px; }
  thead th:hover { color: #fff; background: rgba(255, 255, 255, 0.12); }
  tbody tr { transition: background 0.2s ease; }
  tbody tr:nth-child(even) { background: rgba(255, 255, 255, 0.02); }
  tbody tr:hover { background: rgba(255, 255, 255, 0.06) !important; }
  tbody td { padding: 12px 16px; font-size: 0.9rem; border-bottom: 1px solid rgba(255, 255, 255, 0.05); }

  /* Status Badges */
  .badge-active {
    background: rgba(105, 240, 174, 0.15); color: #69F0AE; border: 1px solid rgba(105, 240, 174, 0.3);
    border-radius: 9999px; padding: 2px 10px; font-size: 0.75rem; font-weight: 600; display: inline-flex; align-items: center; gap: 4px;
  }
  .badge-overdue {
    background: rgba(255, 82, 82, 0.15); color: #FF5252; border: 1px solid rgba(255, 82, 82, 0.3);
    border-radius: 9999px; padding: 2px 10px; font-size: 0.75rem; font-weight: 600; display: inline-flex; align-items: center; gap: 4px;
  }
  .badge-returned {
    background: rgba(255, 255, 255, 0.08); color: #7B6F8A; border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 9999px; padding: 2px 10px; font-size: 0.75rem; font-weight: 500; display: inline-flex; align-items: center; gap: 4px;
  }
  .badge-pending {
    background: rgba(255, 215, 64, 0.15); color: #FFD740; border: 1px solid rgba(255, 215, 64, 0.3);
    border-radius: 9999px; padding: 2px 10px; font-size: 0.75rem; font-weight: 600; display: inline-flex; align-items: center; gap: 4px;
  }

  /* Pulse red glow for overdue badge */
  @keyframes pulseRed { 0%, 100% { box-shadow: 0 0 0 0 rgba(255, 82, 82, 0.5); } 50% { box-shadow: 0 0 0 8px rgba(255, 82, 82, 0); } }
  .pulse-overdue { animation: pulseRed 2s infinite; }

  /* Gradient Text */
  .text-gradient {
    background: linear-gradient(135deg, #E040FB, #448AFF);
    -webkit-background-clip: text; -webkit-text-fill-color: transparent;
  }
  .text-gradient-purple {
    background: linear-gradient(135deg, #CE93D8, #9C27B0);
    -webkit-background-clip: text; -webkit-text-fill-color: transparent;
  }

  .animate-fade-in { animation: fadeIn 0.3s ease forwards; }
  @keyframes fadeIn { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY(0); } }

  /* Sidebar Active */
  .sidebar-item { position: relative; transition: all 0.2s ease; }
  .sidebar-item.active { background: rgba(255, 255, 255, 0.08); color: #ffffff; }
  .sidebar-item.active::before {
    content: ''; position: absolute; left: 0; top: 0; bottom: 0; width: 4px; border-radius: 0 4px 4px 0;
    background: linear-gradient(180deg, #E040FB, #1565C0); box-shadow: 0 0 10px #E040FB;
  }

  /* Print Stylesheet */
  @media print {
    body { background: #fff !important; color: #000 !important; }
    #orbCanvas, .mesh-grid, .glass-sidebar, .glass-header, .page-progress-bar, .no-print { display: none !important; }
    .glass-card { background: transparent !important; border: 1px solid #ccc !important; box-shadow: none !important; backdrop-filter: none !important; color: #000 !important; }
    table thead th { background: #eee !important; color: #000 !important; }
    table tbody td { color: #000 !important; }
  }
</style>
</head>
<body>
  <canvas id="orbCanvas"></canvas>
  <div class="mesh-grid"></div>
  <div id="root"></div>

<script type="text/babel">
const { useState, useEffect, useRef, useMemo, useCallback } = React;

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// DATA CONSTANTS & HELPERS
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
const INITIAL_USERS = [
  { id: 'u1', username: 'librarian', password: 'lib123', name: 'Dr. Sarah Connor', role: 'LIBRARIAN' },
  { id: 'u2', username: 'student1', password: 'mem123', name: 'Ravi Kumar', membershipId: 'M001', memberType: 'Student', maxBooks: 3, role: 'MEMBER', email: 'ravi@library.org' },
  { id: 'u3', username: 'faculty1', password: 'fac123', name: 'Dr. Priya Sharma', membershipId: 'M002', memberType: 'Faculty', maxBooks: 5, role: 'MEMBER', email: 'priya@library.org' },
  { id: 'u4', username: 'admin', password: 'adm123', name: 'System Administrator', role: 'ADMIN' },
];

const INITIAL_BOOKS = [
  { isbn: '978-1', title: 'Effective Java', author: 'Joshua Bloch', category: 'Programming', totalCopies: 3, availableCopies: 3 },
  { isbn: '978-2', title: 'Clean Code', author: 'Robert Martin', category: 'Programming', totalCopies: 2, availableCopies: 2 },
  { isbn: '978-3', title: 'The Pragmatic Programmer', author: 'David Thomas', category: 'Programming', totalCopies: 4, availableCopies: 4 },
  { isbn: '978-4', title: 'Design Patterns', author: 'Gang of Four', category: 'Architecture', totalCopies: 2, availableCopies: 2 },
  { isbn: '978-5', title: 'Introduction to Algorithms', author: 'Cormen', category: 'Computer Science', totalCopies: 5, availableCopies: 5 },
  { isbn: '978-6', title: 'Artificial Intelligence', author: 'Russell & Norvig', category: 'AI', totalCopies: 3, availableCopies: 3 },
  { isbn: '978-7', title: 'Database Systems', author: 'Ramakrishnan', category: 'Database', totalCopies: 2, availableCopies: 2 },
  { isbn: '978-8', title: 'Operating Systems', author: 'Silberschatz', category: 'Systems', totalCopies: 4, availableCopies: 4 },
];

const getTodayStr = () => new Date().toISOString().split('T')[0];
const addDays = (dStr, n) => { const d = new Date(dStr); d.setDate(d.getDate() + n); return d.toISOString().split('T')[0]; };
const getDiffDays = (due, now = getTodayStr()) => Math.ceil((new Date(now) - new Date(due)) / (86400000));

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// ANIMATED CANVAS: 5 FLOATING GRADIENT ORBS
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
function CanvasOrbs() {
  useEffect(() => {
    const canvas = document.getElementById('orbCanvas');
    const ctx = canvas.getContext('2d');
    let w = (canvas.width = window.innerWidth), h = (canvas.height = window.innerHeight);
    const onResize = () => { w = canvas.width = window.innerWidth; h = canvas.height = window.innerHeight; };
    window.addEventListener('resize', onResize);

    const orbs = [
      { x: w * 0.2, y: h * 0.25, r: 340, vx: 0.35, vy: 0.25, color: '#3D1A78' },
      { x: w * 0.8, y: h * 0.3, r: 400, vx: -0.28, vy: 0.3, color: '#1A237E' },
      { x: w * 0.5, y: h * 0.7, r: 360, vx: 0.22, vy: -0.28, color: '#4A148C' },
      { x: w * 0.15, y: h * 0.8, r: 310, vx: 0.3, vy: -0.22, color: '#0D47A1' },
      { x: w * 0.85, y: h * 0.85, r: 350, vx: -0.25, vy: -0.25, color: '#1B0036' }
    ];

    let animId;
    function render() {
      ctx.fillStyle = '#0B0B1A';
      ctx.fillRect(0, 0, w, h);
      orbs.forEach(orb => {
        orb.x += orb.vx; orb.y += orb.vy;
        if (orb.x - orb.r * 0.4 < 0 || orb.x + orb.r * 0.4 > w) orb.vx *= -1;
        if (orb.y - orb.r * 0.4 < 0 || orb.y + orb.r * 0.4 > h) orb.vy *= -1;
        const grad = ctx.createRadialGradient(orb.x, orb.y, 0, orb.x, orb.y, orb.r);
        grad.addColorStop(0, orb.color + 'BF');
        grad.addColorStop(0.5, orb.color + '4D');
        grad.addColorStop(1, 'transparent');
        ctx.fillStyle = grad;
        ctx.beginPath();
        ctx.arc(orb.x, orb.y, orb.r, 0, Math.PI * 2);
        ctx.fill();
      });
      animId = requestAnimationFrame(render);
    }
    render();
    return () => { cancelAnimationFrame(animId); window.removeEventListener('resize', onResize); };
  }, []);
  return null;
}

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// ANIMATED COUNT-UP NUMBER
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
function AnimatedCount({ target, prefix = '', suffix = '' }) {
  const [count, setCount] = useState(0);
  useEffect(() => {
    let start = 0; const end = parseInt(target, 10) || 0;
    if (end === 0) { setCount(0); return; }
    const duration = 750, startTime = performance.now();
    const update = (now) => {
      const progress = Math.min((now - startTime) / duration, 1);
      const ease = 1 - Math.pow(1 - progress, 3);
      setCount(Math.floor(start + (end - start) * ease));
      if (progress < 1) requestAnimationFrame(update); else setCount(end);
    };
    requestAnimationFrame(update);
  }, [target]);
  return <span>{prefix}{count}{suffix}</span>;
}

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// SORTABLE TABLE HOOK
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
function useSortableData(items, config = null) {
  const [sortConfig, setSortConfig] = useState(config);
  const sortedItems = useMemo(() => {
    let sortable = [...items];
    if (sortConfig !== null) {
      sortable.sort((a, b) => {
        let aVal = a[sortConfig.key], bVal = b[sortConfig.key];
        if (typeof aVal === 'string') aVal = aVal.toLowerCase();
        if (typeof bVal === 'string') bVal = bVal.toLowerCase();
        if (aVal < bVal) return sortConfig.direction === 'ascending' ? -1 : 1;
        if (aVal > bVal) return sortConfig.direction === 'ascending' ? 1 : -1;
        return 0;
      });
    }
    return sortable;
  }, [items, sortConfig]);

  const requestSort = (key) => {
    let direction = 'ascending';
    if (sortConfig && sortConfig.key === key && sortConfig.direction === 'ascending') direction = 'descending';
    setSortConfig({ key, direction });
  };
  const getSortIcon = (key) => (!sortConfig || sortConfig.key !== key ? ' â‡…' : sortConfig.direction === 'ascending' ? ' â†‘' : ' â†“');
  return { items: sortedItems, requestSort, getSortIcon };
}

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// FLOATING INPUT COMPONENT
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
function FloatingInput({ label, type = 'text', value, onChange, error, required = false, ...props }) {
  const isValid = value && String(value).trim().length > 0 && !error;
  return (
    <div className="floating-input-wrap">
      <input type={type} value={value} onChange={onChange} placeholder=" " className="floating-input" {...props}/>
      <label className="floating-label">{label} {required && '*'}</label>
      {isValid && <span className="input-valid-check">âœ“</span>}
      {error && <p className="text-xs text-[#FF5252] mt-1 pl-1 font-medium">{error}</p>}
    </div>
  );
}

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// MODAL DIALOG COMPONENT
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
function Modal({ isOpen, onClose, title, children, maxWidth = 'max-w-lg' }) {
  useEffect(() => {
    const onKey = (e) => { if (e.key === 'Escape') onClose(); };
    if (isOpen) window.addEventListener('keydown', onKey);
    return () => window.removeEventListener('keydown', onKey);
  }, [isOpen, onClose]);
  if (!isOpen) return null;
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
      <div className="absolute inset-0 bg-[#0B0B1A]/70 backdrop-blur-md" onClick={onClose}/>
      <div className={`relative w-full ${maxWidth} glass-modal p-6 z-10 overflow-hidden`}>
        <div className="flex items-center justify-between pb-3.5 border-b border-white/10 mb-5">
          <h3 className="font-heading text-lg font-bold text-gradient">{title}</h3>
          <button onClick={onClose} className="w-8 h-8 rounded-full flex items-center justify-center text-[#CE93D8] hover:text-white hover:bg-white/10">âœ•</button>
        </div>
        <div>{children}</div>
      </div>
    </div>
  );
}

// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
// APPLICATION MAIN COMPONENT (THE 8 CORE PAGES)
// 1. Login  2. Dashboard  3. Books  4. Issue Book
// 5. Return Book  6. Members  7. Reservations  8. Reports
// â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰â•⍰
function App() {
  const [currentUser, setCurrentUser] = useState(null);
  const [activeTab, setActiveTab] = useState('dashboard');
  const [sidebarCollapsed, setSidebarCollapsed] = useState(false);
  const [mobileDrawerOpen, setMobileDrawerOpen] = useState(false);
  const [progressWidth, setProgressWidth] = useState(0);

  // Core Data
  const [users, setUsers] = useState(INITIAL_USERS);
  const [books, setBooks] = useState(INITIAL_BOOKS);
  const [loans, setLoans] = useState([]);
  const [reservations, setReservations] = useState([]);
  const [fines, setFines] = useState([]);
  const [recentActivities, setRecentActivities] = useState([
    { id: 1, action: 'System Initialized', detail: 'LibraryOS Dark Glassmorphism 8-Page Edition active', time: 'Just now' }
  ]);
  const [settings, setSettings] = useState({ fineRate: 2, maxBooksDefault: 3 });

  // ID Counters
  const [loanCounter, setLoanCounter] = useState(1000);
  const [fineCounter, setFineCounter] = useState(100);
  const [reservationCounter, setReservationCounter] = useState(100);

  // Toasts
  const [toasts, setToasts] = useState([]);
  const showToast = useCallback((message, type = 'success', title = '') => {
    const id = Date.now() + Math.random();
    setToasts((p) => [...p, { id, message, type, title }]);
    setTimeout(() => setToasts((p) => p.filter((t) => t.id !== id)), 3000);
  }, []);

  const logActivity = (action, detail) => {
    setRecentActivities((p) => [
      { id: Date.now(), action, detail, time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }) },
      ...p.slice(0, 4)
    ]);
  };

  const switchTab = (tab) => {
    setProgressWidth(35);
    setTimeout(() => setProgressWidth(75), 90);
    setTimeout(() => {
      setActiveTab(tab); setProgressWidth(100); setMobileDrawerOpen(false);
      setTimeout(() => setProgressWidth(0), 220);
    }, 180);
  };

  const handleLogin = (user) => {
    setCurrentUser(user);
    setActiveTab('dashboard');
    showToast(`Welcome back, ${user.name}!`, 'success', 'Terminal Access');
    logActivity('User Access', `${user.name} logged in`);
  };

  const handleLogout = () => {
    setCurrentUser(null);
    showToast('Session ended successfully.', 'warning');
  };

  const overdueCount = useMemo(() => {
    const today = getTodayStr();
    return loans.filter((l) => l.status === 'ACTIVE' && l.dueDate < today).length;
  }, [loans]);

  // â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  // MODAL STATES
  // â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  const [bookModalOpen, setBookModalOpen] = useState(false);
  const [editingBook, setEditingBook] = useState(null);
  const [bookForm, setBookForm] = useState({ isbn: '', title: '', author: '', category: '', copies: 1 });
  const [bookErrors, setBookErrors] = useState({});

  const [issueMemberId, setIssueMemberId] = useState('');
  const [issueBookIsbn, setIssueBookIsbn] = useState('');

  const [returnModalOpen, setReturnModalOpen] = useState(false);
  const [selectedLoanReturn, setSelectedLoanReturn] = useState(null);

  const [memberModalOpen, setMemberModalOpen] = useState(false);
  const [memberForm, setMemberForm] = useState({ name: '', username: '', password: '', memberType: 'Student', email: '' });
  const [memberErrors, setMemberErrors] = useState({});

  // Search Filters
  const [bookSearch, setBookSearch] = useState('');
  const [memberSearch, setMemberSearch] = useState('');
  const [returnSearch, setReturnSearch] = useState('');

  // â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  // HANDLERS
  // â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
  const handleSaveBook = (e) => {
    e.preventDefault();
    const errs = {};
    if (!bookForm.isbn.trim()) errs.isbn = 'ISBN required';
    if (!bookForm.title.trim()) errs.title = 'Title required';
    if (!bookForm.author.trim()) errs.author = 'Author required';
    if (!bookForm.category.trim()) errs.category = 'Category required';
    if (!bookForm.copies || bookForm.copies < 1) errs.copies = 'Minimum 1 copy';
    if (Object.keys(errs).length) { setBookErrors(errs); return; }

    if (editingBook) {
      const diff = parseInt(bookForm.copies, 10) - editingBook.totalCopies;
      setBooks((prev) => prev.map((b) => b.isbn === editingBook.isbn ? {
        ...b, title: bookForm.title, author: bookForm.author, category: bookForm.category,
        totalCopies: parseInt(bookForm.copies, 10), availableCopies: Math.max(0, b.availableCopies + diff)
      } : b));
      showToast(`Updated "${bookForm.title}" successfully.`);
      logActivity('Book Updated', `Modified copies of ${bookForm.title}`);
    } else {
      if (books.some((b) => b.isbn === bookForm.isbn.trim())) { setBookErrors({ isbn: 'ISBN already exists' }); return; }
      setBooks((prev) => [{
        isbn: bookForm.isbn.trim(), title: bookForm.title.trim(), author: bookForm.author.trim(),
        category: bookForm.category.trim(), totalCopies: parseInt(bookForm.copies, 10), availableCopies: parseInt(bookForm.copies, 10)
      }, ...prev]);
      showToast(`Added "${bookForm.title}" to catalogue.`);
      logActivity('Book Added', `New book ${bookForm.title} added`);
    }
    setBookModalOpen(false);
  };

  const handleDeleteBook = (isbn) => {
    if (loans.some((l) => l.bookIsbn === isbn && l.status === 'ACTIVE')) {
      showToast('Cannot delete book with active loans.', 'danger'); return;
    }
    const b = books.find((x) => x.isbn === isbn);
    setBooks((prev) => prev.filter((x) => x.isbn !== isbn));
    showToast(`Removed "${b?.title}".`, 'warning');
    logActivity('Book Deleted', `Deleted ${b?.title}`);
  };

  const handleIssueBook = (e) => {
    e.preventDefault();
    if (!issueMemberId || !issueBookIsbn) { showToast('Select both member and book.', 'warning'); return; }
    const m = users.find((u) => u.id === issueMemberId);
    const b = books.find((x) => x.isbn === issueBookIsbn);
    if (!m || !b) return;

    if (b.availableCopies <= 0) { showToast(`No copies available for "${b.title}".`, 'danger'); return; }
    const activeCount = loans.filter((l) => l.memberId === m.id && l.status === 'ACTIVE').length;
    const limit = m.maxBooks || settings.maxBooksDefault;
    if (activeCount >= limit) { showToast(`${m.name} reached max limit of ${limit} books!`, 'danger'); return; }

    const loanId = `LN${loanCounter}`; setLoanCounter((c) => c + 1);
    const today = getTodayStr(), dueDate = addDays(today, 15);
    setLoans((prev) => [{
      id: loanId, memberId: m.id, memberName: m.name, membershipId: m.membershipId,
      bookIsbn: b.isbn, bookTitle: b.title, issueDate: today, dueDate, status: 'ACTIVE'
    }, ...prev]);
    setBooks((prev) => prev.map((x) => x.isbn === b.isbn ? { ...x, availableCopies: x.availableCopies - 1 } : x));
    showToast(`Issued "${b.title}" to ${m.name}! Due: ${dueDate}`, 'success');
    logActivity('Book Issued', `${loanId}: ${b.title} issued to ${m.name}`);
    setIssueMemberId(''); setIssueBookIsbn('');
  };

  const handleConfirmReturn = (payFineImmediately = false) => {
    if (!selectedLoanReturn) return;
    const loan = selectedLoanReturn, today = getTodayStr();
    const overdueDays = getDiffDays(loan.dueDate, today);
    const fine = overdueDays > 0 ? overdueDays * settings.fineRate : 0;

    if (fine > 0) {
      const fineId = `FN${fineCounter}`; setFineCounter((c) => c + 1);
      setFines((prev) => [{
        id: fineId, loanId: loan.id, memberId: loan.memberId, memberName: loan.memberName,
        bookTitle: loan.bookTitle, amount: fine, status: payFineImmediately ? 'SETTLED' : 'UNPAID', date: today
      }, ...prev]);
    }
    setLoans((prev) => prev.map((l) => l.id === loan.id ? { ...l, status: 'RETURNED', returnDate: today } : l));
    setBooks((prev) => prev.map((b) => b.isbn === loan.bookIsbn ? { ...b, availableCopies: b.availableCopies + 1 } : b));

    const pendingRes = reservations.find((r) => r.bookIsbn === loan.bookIsbn && r.status === 'PENDING');
    if (pendingRes) showToast(`Pending reservation exists for "${loan.bookTitle}" by ${pendingRes.memberName}!`, 'warning', 'Hold Alert');
    else showToast(`"${loan.bookTitle}" returned successfully!`, 'success');

    logActivity('Book Returned', `Loan ${loan.id} returned`);
    setReturnModalOpen(false); setSelectedLoanReturn(null);
  };

  const handleRegisterMember = (e) => {
    e.preventDefault();
    const errs = {};
    if (!memberForm.name.trim()) errs.name = 'Full name required';
    if (!memberForm.username.trim()) errs.username = 'Username required';
    if (!memberForm.password.trim() || memberForm.password.length < 4) errs.password = 'Min 4 characters';
    if (!memberForm.email.trim() || !memberForm.email.includes('@')) errs.email = 'Valid email required';
    if (users.some((u) => u.username.toLowerCase() === memberForm.username.trim().toLowerCase())) errs.username = 'Username taken';
    if (Object.keys(errs).length) { setMemberErrors(errs); return; }

    const num = users.filter((u) => u.role === 'MEMBER').length + 1;
    const membershipId = `M00${num}`;
    const newMember = {
      id: `u_${Date.now()}`, username: memberForm.username.trim(), password: memberForm.password.trim(),
      name: memberForm.name.trim(), membershipId, memberType: memberForm.memberType,
      maxBooks: memberForm.memberType === 'Faculty' ? 5 : 3, role: 'MEMBER', email: memberForm.email.trim()
    };
    setUsers((p) => [...p, newMember]);
    showToast(`Registered member ${newMember.name} (${membershipId})!`);
    logActivity('Member Registered', `${newMember.name} joined`);
    setMemberModalOpen(false);
    setMemberForm({ name: '', username: '', password: '', memberType: 'Student', email: '' });
  };

  const handleReserveBook = (book) => {
    const existing = reservations.find((r) => r.bookIsbn === book.isbn && r.status === 'PENDING');
    if (existing) { showToast(`A reservation is already placed for "${book.title}".`, 'warning'); return; }
    const resId = `RV${reservationCounter}`; setReservationCounter((c) => c + 1);
    setReservations((prev) => [{
      id: resId, memberId: currentUser?.id || 'guest', memberName: currentUser?.name || 'Dr. Sarah Connor',
      bookIsbn: book.isbn, bookTitle: book.title, date: getTodayStr(), status: 'PENDING'
    }, ...prev]);
    showToast(`Reserved "${book.title}" (${resId})!`, 'success');
    logActivity('Book Reserved', `Reservation ${resId} created for ${book.title}`);
  };

  const handleCancelReservation = (id) => {
    setReservations((prev) => prev.map((r) => r.id === id ? { ...r, status: 'CANCELLED' } : r));
    showToast('Reservation cancelled.', 'warning');
    logActivity('Reservation Cancelled', `${id} cancelled`);
  };

  // Filtered Items
  const filteredBooks = useMemo(() => books.filter((b) =>
    b.title.toLowerCase().includes(bookSearch.toLowerCase()) ||
    b.author.toLowerCase().includes(bookSearch.toLowerCase()) ||
    b.category.toLowerCase().includes(bookSearch.toLowerCase()) ||
    b.isbn.includes(bookSearch)
  ), [books, bookSearch]);
  const { items: sortedBooks, requestSort: sortBooks, getSortIcon: getBookSortIcon } = useSortableData(filteredBooks);

  const filteredMembers = useMemo(() => users.filter((u) => u.role === 'MEMBER').filter((m) =>
    m.name.toLowerCase().includes(memberSearch.toLowerCase()) ||
    m.membershipId?.toLowerCase().includes(memberSearch.toLowerCase())
  ), [users, memberSearch]);
  const { items: sortedMembers, requestSort: sortMembers, getSortIcon: getMemberSortIcon } = useSortableData(filteredMembers);

  const activeLoansList = useMemo(() => loans.filter((l) => l.status === 'ACTIVE').filter((l) =>
    l.bookTitle.toLowerCase().includes(returnSearch.toLowerCase()) ||
    l.memberName.toLowerCase().includes(returnSearch.toLowerCase()) ||
    l.id.toLowerCase().includes(returnSearch.toLowerCase())
  ), [loans, returnSearch]);
  const { items: sortedActiveLoans, requestSort: sortActiveLoans, getSortIcon: getLoanSortIcon } = useSortableData(activeLoansList);

  const { items: sortedReservations, requestSort: sortReservations, getSortIcon: getResSortIcon } = useSortableData(reservations);

  // App Navigation: 7 Core Management Pages (+ Login = 8 total)
  const navItems = [
    { id: 'dashboard', label: 'Dashboard', icon: 'ðŸ“Š' },
    { id: 'books', label: 'Manage Books', icon: 'ðŸ“š' },
    { id: 'issue', label: 'Issue Book', icon: 'ðŸ“¤' },
    { id: 'return', label: 'Return Book', icon: 'ðŸ“¥', badge: overdueCount > 0 ? overdueCount : null },
    { id: 'members', label: 'Manage Members', icon: 'ðŸ‘¥' },
    { id: 'reservations', label: 'Reservations', icon: 'ðŸ”–' },
    { id: 'reports', label: 'Reports & Audit', icon: 'ðŸ“ˆ' }
  ];

  return (
    <div className="relative w-full h-full flex flex-col overflow-hidden select-none">
      <CanvasOrbs />
      <div className="page-progress-bar" style={{ width: `${progressWidth}%`, opacity: progressWidth > 0 ? 1 : 0 }}/>

      {/* Toasts */}
      <div className="fixed top-5 right-5 z-50 flex flex-col gap-2.5 pointer-events-none max-w-sm w-full">
        {toasts.map((t) => (
          <div key={t.id} className="pointer-events-auto p-4 rounded-2xl glass-card flex items-center justify-between shadow-2xl border"
            style={{
              borderColor: t.type === 'success' ? 'rgba(105, 240, 174, 0.4)' : t.type === 'danger' ? 'rgba(255, 82, 82, 0.4)' : 'rgba(255, 215, 64, 0.4)',
              background: 'rgba(20, 15, 35, 0.95)'
            }}>
            <div className="flex items-center gap-3">
              <span className="text-xl">{t.type === 'success' ? 'âœ¨' : t.type === 'danger' ? 'âš ï¸⍰' : 'ðŸ””'}</span>
              <div>
                <p className="text-xs font-bold" style={{ color: t.type === 'success' ? '#69F0AE' : t.type === 'danger' ? '#FF5252' : '#FFD740' }}>{t.title || 'Notice'}</p>
                <p className="text-xs text-[#F3E5F5]/90">{t.message}</p>
              </div>
            </div>
            <button onClick={() => setToasts((prev) => prev.filter((x) => x.id !== t.id))} className="text-white/60 hover:text-white ml-2 text-base">âœ•</button>
          </div>
        ))}
      </div>

      {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
          PAGE 1: LOGIN SCREEN
      â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
      {!currentUser ? (
        <div className="relative z-10 w-full h-full flex items-center justify-center p-4">
          <div className="w-full max-w-md glass-modal p-8 md:p-10 text-center animate-fade-in shadow-2xl">
            <div className="w-16 h-16 mx-auto mb-4 rounded-2xl bg-gradient-to-tr from-[#9C27B0]/30 to-[#1565C0]/30 border border-white/20 flex items-center justify-center text-3xl shadow-xl shadow-[#9C27B0]/20">
              ðŸ“–
            </div>
            <h1 className="font-heading text-3xl font-extrabold text-gradient mb-1">LibraryOS</h1>
            <p className="text-xs font-semibold text-[#CE93D8] tracking-wider uppercase mb-7">Dark Glassmorphism Terminal</p>

            <form onSubmit={(e) => {
              e.preventDefault();
              const u = e.target.username.value.trim(), p = e.target.password.value.trim();
              const found = users.find((x) => x.username.toLowerCase() === u.toLowerCase() && x.password === p);
              if (found) handleLogin(found); else showToast('Invalid credentials. Check username/password.', 'danger');
            }} className="text-left">
              <FloatingInput name="username" label="Username" defaultValue="librarian" required />
              <FloatingInput name="password" label="Password" type="password" defaultValue="lib123" required />
              <button type="submit" className="btn-primary w-full py-3.5 text-xs font-bold uppercase tracking-wider mt-2 mb-6">
                Enter Terminal
              </button>
            </form>

            <div className="mb-5">
              <p className="text-[10px] text-[#7B6F8A] font-bold uppercase tracking-wider mb-2">Quick Sign-In Presets</p>
              <div className="flex items-center justify-center gap-2">
                <button onClick={() => handleLogin(users[0])} className="btn-glass-pill px-3 py-1.5 text-xs text-[#CE93D8] hover:text-white">ðŸ‘©â€⍰ðŸ’¼ Librarian</button>
                <button onClick={() => handleLogin(users[1])} className="btn-glass-pill px-3 py-1.5 text-xs text-[#CE93D8] hover:text-white">ðŸŽ“ Member</button>
                <button onClick={() => handleLogin(users[3])} className="btn-glass-pill px-3 py-1.5 text-xs text-[#CE93D8] hover:text-white">ðŸ›¡ï¸⍰ Admin</button>
              </div>
            </div>
            <p className="text-[10px] text-[#7B6F8A]">ðŸ”’ Single-Page Reactive Data Store â€¢ 8 Core Pages</p>
          </div>
        </div>
      ) : (
        <div className="relative z-10 flex h-full w-full overflow-hidden">
          {/* Mobile Backdrop */}
          {mobileDrawerOpen && <div className="fixed inset-0 bg-[#0B0B1A]/80 backdrop-blur-sm z-30 md:hidden" onClick={() => setMobileDrawerOpen(false)}/>}

          {/* Sidebar Navigation */}
          <aside className={`glass-sidebar fixed md:relative z-40 h-full flex flex-col justify-between transition-all duration-300 ${
            mobileDrawerOpen ? 'left-0 w-64' : '-left-full md:left-0'
          } ${sidebarCollapsed ? 'md:w-[70px]' : 'md:w-[260px]'}`}>
            <div>
              <div className="h-16 flex items-center justify-between px-4 border-b border-white/10">
                {!sidebarCollapsed ? (
                  <div className="flex items-center gap-2">
                    <span className="text-2xl drop-shadow-[0_0_8px_rgba(224,64,251,0.6)]">ðŸ“–</span>
                    <span className="font-heading font-extrabold text-base text-gradient">LibraryOS</span>
                  </div>
                ) : (
                  <span className="text-2xl mx-auto">ðŸ“–</span>
                )}
                <button onClick={() => setSidebarCollapsed(!sidebarCollapsed)} className="hidden md:flex w-7 h-7 rounded-lg bg-white/5 hover:bg-white/15 text-[#CE93D8] hover:text-white items-center justify-center text-xs">
                  {sidebarCollapsed ? 'â–¶' : 'â—€'}
                </button>
              </div>

              <nav className="p-3 space-y-1 overflow-y-auto">
                {navItems.map((item) => (
                  <button key={item.id} onClick={() => switchTab(item.id)}
                    className={`w-full flex items-center gap-3.5 px-3.5 py-3 rounded-xl sidebar-item font-medium text-xs transition-all ${
                      activeTab === item.id ? 'active' : 'text-[#CE93D8] hover:text-white hover:bg-white/5'
                    }`}>
                    <span className="text-base shrink-0">{item.icon}</span>
                    {!sidebarCollapsed && <span className="truncate">{item.label}</span>}
                    {!sidebarCollapsed && item.badge && (
                      <span className="ml-auto px-2 py-0.5 rounded-full text-[10px] font-bold bg-[#FF5252] text-white pulse-overdue">{item.badge}</span>
                    )}
                  </button>
                ))}
              </nav>
            </div>

            <div className="p-3 border-t border-white/10">
              <div className="glass-card p-2.5 flex items-center gap-3 rounded-xl">
                <div className="w-8 h-8 shrink-0 rounded-full bg-gradient-to-tr from-[#9C27B0] to-[#1565C0] flex items-center justify-center font-bold text-xs text-white">
                  {currentUser.name.slice(0, 2).toUpperCase()}
                </div>
                {!sidebarCollapsed && (
                  <div className="overflow-hidden">
                    <p className="text-xs font-bold text-white truncate">{currentUser.name}</p>
                    <span className="badge-active text-[9px] py-0 px-2 mt-0.5">{currentUser.role}</span>
                  </div>
                )}
              </div>
            </div>
          </aside>

          {/* Main Workspace */}
          <div className="flex-1 flex flex-col h-full overflow-hidden">
            <header className="glass-header h-16 w-full flex items-center justify-between px-4 md:px-8 z-20 shrink-0">
              <div className="flex items-center gap-3">
                <button onClick={() => setMobileDrawerOpen(!mobileDrawerOpen)} className="md:hidden text-[#CE93D8] p-2 rounded-lg bg-white/5">â˜°</button>
                <div className="font-heading text-sm md:text-base font-bold text-white">
                  {navItems.find((n) => n.id === activeTab)?.label}
                </div>
              </div>
              <div className="flex items-center gap-4">
                <button onClick={handleLogout} className="btn-glass-pill px-3.5 py-1.5 text-xs font-semibold flex items-center gap-1.5">
                  <span>Logout</span> <span>â†³</span>
                </button>
              </div>
            </header>

            <main className="flex-1 overflow-y-auto p-4 md:p-8 animate-fade-in">
              {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                  PAGE 2: DASHBOARD
              â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
              {activeTab === 'dashboard' && (
                <div className="space-y-6">
                  <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-5">
                    <div className="glass-card glass-card-hover p-6 rounded-2xl flex flex-col justify-between">
                      <div className="flex items-center justify-between text-[#CE93D8]">
                        <span className="text-xs font-bold uppercase">Total Books</span><span className="text-xl">ðŸ“š</span>
                      </div>
                      <p className="font-heading text-4xl font-extrabold text-gradient my-2">
                        <AnimatedCount target={books.reduce((a, b) => a + b.totalCopies, 0)} />
                      </p>
                      <span className="text-[11px] text-[#7B6F8A]">Copies in library system</span>
                    </div>

                    <div className="glass-card glass-card-hover p-6 rounded-2xl flex flex-col justify-between">
                      <div className="flex items-center justify-between text-[#CE93D8]">
                        <span className="text-xs font-bold uppercase">Available Copies</span><span className="text-xl">âœ…</span>
                      </div>
                      <p className="font-heading text-4xl font-extrabold text-[#69F0AE] my-2">
                        <AnimatedCount target={books.reduce((a, b) => a + b.availableCopies, 0)} />
                      </p>
                      <span className="text-[11px] text-[#7B6F8A]">Ready for checkout</span>
                    </div>

                    <div className="glass-card glass-card-hover p-6 rounded-2xl flex flex-col justify-between">
                      <div className="flex items-center justify-between text-[#CE93D8]">
                        <span className="text-xs font-bold uppercase">Active Loans</span><span className="text-xl">ðŸ“¤</span>
                      </div>
                      <p className="font-heading text-4xl font-extrabold text-[#448AFF] my-2">
                        <AnimatedCount target={loans.filter((l) => l.status === 'ACTIVE').length} />
                      </p>
                      <span className="text-[11px] text-[#7B6F8A]">Currently issued to patrons</span>
                    </div>

                    <div className="glass-card glass-card-hover p-6 rounded-2xl flex flex-col justify-between">
                      <div className="flex items-center justify-between text-[#CE93D8]">
                        <span className="text-xs font-bold uppercase">Overdue Loans</span><span className="text-xl">âš ï¸⍰</span>
                      </div>
                      <p className={`font-heading text-4xl font-extrabold my-2 ${overdueCount > 0 ? 'text-[#FF5252]' : 'text-white'}`}>
                        <AnimatedCount target={overdueCount} />
                      </p>
                      <span className="text-[11px] text-[#7B6F8A]">{overdueCount > 0 ? 'Action required' : 'All accounts in good standing'}</span>
                    </div>

                    <div className="glass-card glass-card-hover p-6 rounded-2xl flex flex-col justify-between">
                      <div className="flex items-center justify-between text-[#CE93D8]">
                        <span className="text-xs font-bold uppercase">Total Members</span><span className="text-xl">ðŸ‘¥</span>
                      </div>
                      <p className="font-heading text-4xl font-extrabold text-gradient-purple my-2">
                        <AnimatedCount target={users.filter((u) => u.role === 'MEMBER').length} />
                      </p>
                      <span className="text-[11px] text-[#7B6F8A]">Students & Faculty enrolled</span>
                    </div>

                    <div className="glass-card glass-card-hover p-6 rounded-2xl flex flex-col justify-between">
                      <div className="flex items-center justify-between text-[#CE93D8]">
                        <span className="text-xs font-bold uppercase">Fines Settled</span><span className="text-xl">ðŸ’°</span>
                      </div>
                    <p className="font-heading text-4xl font-extrabold text-[#FFD740] my-2">
                      <AnimatedCount target={fines.filter((f) => f.status === 'SETTLED').reduce((a, b) => a + b.amount, 0)} prefix="â‚¹" />
                    </p>
                    <span className="text-[11px] text-[#7B6F8A]">Revenue recovered</span>
                  </div>
                </div>

                <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
                  <div className="lg:col-span-2 glass-card p-6 rounded-2xl">
                    <h3 className="font-heading text-sm font-bold text-gradient mb-3 flex items-center gap-2"><span>âš¡</span> Live Activity Stream</h3>
                    <div className="space-y-2.5">
                      {recentActivities.map((act) => (
                        <div key={act.id} className="p-3 rounded-xl bg-white/[0.03] border border-white/5 flex items-center justify-between">
                          <div>
                            <p className="text-xs font-bold text-white">{act.action}</p>
                            <p className="text-[11px] text-[#CE93D8]/80">{act.detail}</p>
                          </div>
                          <span className="text-[10px] text-[#7B6F8A] font-semibold">{act.time}</span>
                        </div>
                      ))}
                    </div>
                  </div>
                  <div className="glass-card p-6 rounded-2xl flex flex-col justify-between">
                    <div>
                      <h3 className="font-heading text-sm font-bold text-gradient mb-3 flex items-center gap-2"><span>ðŸš€</span> Quick Shortcuts</h3>
                      <div className="space-y-2.5">
                        <button onClick={() => switchTab('issue')} className="btn-primary w-full py-2.5 text-xs font-bold flex items-center justify-center gap-2">
                          <span>ðŸ“¤</span> Issue Book
                        </button>
                        <button onClick={() => switchTab('return')} className="btn-secondary w-full py-2.5 text-xs font-bold flex items-center justify-center gap-2">
                          <span>ðŸ“¥</span> Return Book
                        </button>
                        <button onClick={() => { setEditingBook(null); setBookForm({ isbn: '', title: '', author: '', category: '', copies: 1 }); setBookErrors({}); setBookModalOpen(true); }}
                          className="btn-glass-pill w-full py-2 text-xs font-bold flex items-center justify-center gap-2">
                          <span>âž•</span> Add New Book
                        </button>
                      </div>
                    </div>
                    <p className="text-[11px] text-[#7B6F8A] text-center mt-4">Overdue Fine Rate: â‚¹{settings.fineRate}/day</p>
                  </div>
                </div>
              </div>
            )}

            {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                PAGE 3: MANAGE BOOKS
            â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
            {activeTab === 'books' && (
              <div className="glass-card p-6 rounded-2xl space-y-5">
                <div className="flex flex-col sm:flex-row items-center justify-between gap-4">
                  <div>
                    <h2 className="font-heading text-lg font-bold text-gradient">Book Inventory Management</h2>
                    <p className="text-xs text-[#7B6F8A]">Live cataloguing, copy adjustment, and title search</p>
                  </div>
                  <div className="flex items-center gap-3 w-full sm:w-auto">
                    <input type="text" placeholder="Search title, author, category..." value={bookSearch} onChange={(e) => setBookSearch(e.target.value)}
                      className="w-full sm:w-64 px-4 py-2 text-xs rounded-xl bg-white/[0.06] border border-white/10 text-white outline-none focus:border-[#E040FB]"/>
                    <button onClick={() => { setEditingBook(null); setBookForm({ isbn: '', title: '', author: '', category: '', copies: 1 }); setBookErrors({}); setBookModalOpen(true); }}
                      className="btn-primary py-2 px-4 text-xs font-bold whitespace-nowrap">+ Add Book</button>
                  </div>
                </div>

                <div className="overflow-x-auto rounded-xl">
                  <table>
                    <thead>
                      <tr>
                        <th onClick={() => sortBooks('isbn')}>ISBN {getBookSortIcon('isbn')}</th>
                        <th onClick={() => sortBooks('title')}>Title {getBookSortIcon('title')}</th>
                        <th onClick={() => sortBooks('author')}>Author {getBookSortIcon('author')}</th>
                        <th onClick={() => sortBooks('category')}>Category {getBookSortIcon('category')}</th>
                        <th onClick={() => sortBooks('totalCopies')}>Total {getBookSortIcon('totalCopies')}</th>
                        <th onClick={() => sortBooks('availableCopies')}>Available {getBookSortIcon('availableCopies')}</th>
                        <th className="text-right">Actions</th>
                      </tr>
                    </thead>
                    <tbody>
                      {sortedBooks.map((b) => (
                        <tr key={b.isbn}>
                          <td className="font-mono text-xs text-[#CE93D8]">{b.isbn}</td>
                          <td className="font-semibold text-white">{b.title}</td>
                          <td className="text-[#CE93D8]/90">{b.author}</td>
                          <td><span className="px-2.5 py-0.5 rounded-full text-[10px] font-semibold bg-white/10 text-[#CE93D8]">{b.category}</span></td>
                          <td className="font-bold text-center">{b.totalCopies}</td>
                          <td className="text-center">
                            <span className={b.availableCopies > 0 ? 'badge-active' : 'badge-overdue'}>{b.availableCopies} left</span>
                          </td>
                          <td className="text-right space-x-2">
                            <button onClick={() => { setEditingBook(b); setBookForm({ isbn: b.isbn, title: b.title, author: b.author, category: b.category, copies: b.totalCopies }); setBookErrors({}); setBookModalOpen(true); }}
                              className="btn-secondary px-2.5 py-1 text-xs">Edit</button>
                            <button onClick={() => handleDeleteBook(b.isbn)} className="btn-danger px-2.5 py-1 text-xs">Delete</button>
                          </td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
              </div>
            )}

            {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                PAGE 4: ISSUE BOOK
            â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
*/}
            {activeTab === 'issue' && (
              <div className="max-w-2xl mx-auto glass-card p-8 rounded-2xl space-y-6">
                <div>
                  <h2 className="font-heading text-lg font-bold text-gradient">Issue Book</h2>
                  <p className="text-xs text-[#7B6F8A]">Select patron and verify availability before loan generation</p>
                </div>
                <form onSubmit={handleIssueBook} className="space-y-4">
                  <div>
                    <label className="block text-xs font-semibold text-[#CE93D8] mb-1">Select Member *</label>
                    <select value={issueMemberId} onChange={(e) => setIssueMemberId(e.target.value)} className="glass-select" required>
                      <option value="">-- Choose Member --</option>
                      {users.filter((u) => u.role === 'MEMBER').map((m) => (
                        <option key={m.id} value={m.id}>{m.name} ({m.membershipId}) â€” {m.memberType}</option>
                      ))}
                    </select>
                  </div>
                  <div>
                    <label className="block text-xs font-semibold text-[#CE93D8] mb-1">Select Available Book *</label>
                    <select value={issueBookIsbn} onChange={(e) => setIssueBookIsbn(e.target.value)} className="glass-select" required>
                      <option value="">-- Choose Book --</option>
                      {books.filter((b) => b.availableCopies > 0).map((b) => (
                        <option key={b.isbn} value={b.isbn}>{b.title} by {b.author} ({b.availableCopies} available)</option>
                      ))}
                    </select>
                  </div>

                  {issueMemberId && issueBookIsbn && (() => {
                    const m = users.find((u) => u.id === issueMemberId);
                    const b = books.find((x) => x.isbn === issueBookIsbn);
                    return (
                      <div className="p-4 rounded-xl bg-white/[0.03] border border-white/10 flex items-center justify-between gap-4 animate-fade-in">
                        <div>
                          <span className="text-[10px] text-[#7B6F8A] uppercase font-bold">Borrower</span>
                          <p className="text-sm font-bold text-white">{m?.name}</p>
                          <p className="text-xs text-[#CE93D8]">{m?.membershipId} ({m?.memberType})</p>
                        </div>
                        <div className="text-2xl text-[#E040FB] animate-pulse">âž”</div>
                        <div className="text-right">
                          <span className="text-[10px] text-[#7B6F8A] uppercase font-bold">Book</span>
                          <p className="text-sm font-bold text-white">{b?.title}</p>
                          <p className="text-xs text-[#CE93D8]">15 Days Loan Quota</p>
                        </div>
                      </div>
                    );
                  })()}

                  <button type="submit" className="btn-primary w-full py-3.5 text-xs font-bold uppercase tracking-wider mt-4">
                    Issue Book Now
                  </button>
                </form>
              </div>
            )}

            {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                PAGE 5: RETURN BOOK
            â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
*/}
            {activeTab === 'return' && (
              <div className="glass-card p-6 rounded-2xl space-y-5">
                <div className="flex flex-col sm:flex-row items-center justify-between gap-4">
                  <div>
                    <h2 className="font-heading text-lg font-bold text-gradient">Return Book Processing</h2>
                    <p className="text-xs text-[#7B6F8A]">Active loans, live due countdown, and fine calculation</p>
                  </div>
                  <input type="text" placeholder="Search loans..." value={returnSearch} onChange={(e) => setReturnSearch(e.target.value)}
                    className="w-full sm:w-64 px-4 py-2 text-xs rounded-xl bg-white/[0.06] border border-white/10 text-white outline-none focus:border-[#E040FB]"/>
                </div>

                <div className="overflow-x-auto rounded-xl">
                  {sortedActiveLoans.length === 0 ? (
                    <div className="p-12 text-center text-[#7B6F8A]"><span className="text-4xl block mb-2">ðŸ“¥</span>No active loans currently checked out.</div>
                  ) : (
                    <table>
                      <thead>
                        <tr>
                          <th onClick={() => sortActiveLoans('id')}>Loan ID {getLoanSortIcon('id')}</th>
                          <th onClick={() => sortActiveLoans('memberName')}>Member {getLoanSortIcon('memberName')}</th>
                          <th onClick={() => sortActiveLoans('bookTitle')}>Book Title {getLoanSortIcon('bookTitle')}</th>
                          <th onClick={() => sortActiveLoans('issueDate')}>Issued {getLoanSortIcon('issueDate')}</th>
                          <th onClick={() => sortActiveLoans('dueDate')}>Due Date {getLoanSortIcon('dueDate')}</th>
                          <th>Days Left</th>
                          <th>Fine (â‚¹2/day)</th>
                          <th className="text-right">Action</th>
                        </tr>
                      </thead>
                      <tbody>
                        {sortedActiveLoans.map((loan) => {
                          const today = getTodayStr(), overdueDays = getDiffDays(loan.dueDate, today);
                          const isOverdue = overdueDays > 0;
                          const fine = isOverdue ? overdueDays * settings.fineRate : 0;
                          return (
                            <tr key={loan.id} className={isOverdue ? 'bg-red-500/[0.06]' : ''}>
                              <td className="font-mono text-xs text-[#CE93D8]">{loan.id}</td>
                              <td className="font-semibold text-white">{loan.memberName}</td>
                              <td className="text-white truncate max-w-xs">{loan.bookTitle}</td>
                              <td className="text-xs text-[#7B6F8A]">{loan.issueDate}</td>
                              <td className="text-xs font-semibold text-white">{loan.dueDate}</td>
```

```
                                  <td>{isOverdue ? <span className="badge-overdue">{overdueDays}d OVERDUE</span> : <span className="badge-active">{Math.abs(overdueDays)}d left</span>}</td>
                                  <td className="font-bold">{fine > 0 ? <span className="text-[#FF5252]">â‚¹{fine}</span> : <span className="text-[#69F0AE]">â‚¹0</span>}</td>
                                  <td className="text-right">
                                    <button onClick={() => { setSelectedLoanReturn(loan); setReturnModalOpen(true); }} className="btn-glass-pill px-3 py-1 text-xs font-semibold">
                                      Return
                                    </button>
                                  </td>
                                </tr>
                              );
                            })}
                          </tbody>
                        </table>
                      )}
                    </div>
                  </div>
                )}

                {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                    PAGE 6: MANAGE MEMBERS
                    â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
                {activeTab === 'members' && (
                  <div className="glass-card p-6 rounded-2xl space-y-5">
                    <div className="flex flex-col sm:flex-row items-center justify-between gap-4">
                      <div>
                        <h2 className="font-heading text-lg font-bold text-gradient">Member Directory</h2>
                        <p className="text-xs text-[#7B6F8A]">Patron account status, borrow quota, and fine balances</p>
                      </div>
                      <div className="flex items-center gap-3 w-full sm:w-auto">
                        <input type="text" placeholder="Search members..." value={memberSearch} onChange={(e) => setMemberSearch(e.target.value)}
                          className="w-full sm:w-64 px-4 py-2 text-xs rounded-xl bg-white/[0.06] border border-white/10 text-white outline-none focus:border-[#E040FB]"/>
                        <button onClick={() => { setMemberErrors({}); setMemberModalOpen(true); }} className="btn-primary py-2 px-4 text-xs font-bold whitespace-nowrap">+ Register Member</button>
                      </div>
                    </div>

                    <div className="overflow-x-auto rounded-xl">
                      <table>
                        <thead>
                          <tr>
                            <th>Avatar</th>
                            <th onClick={() => sortMembers('name')}>Name {getMemberSortIcon('name')}</th>
                            <th onClick={() => sortMembers('membershipId')}>ID {getMemberSortIcon('membershipId')}</th>
                            <th onClick={() => sortMembers('memberType')}>Type {getMemberSortIcon('memberType')}</th>
                            <th>Active Loans</th>
                            <th>Unpaid Fines</th>
                          </tr>
                        </thead>
                        <tbody>
                          {sortedMembers.map((m) => {
                            const active = loans.filter((l) => l.memberId === m.id && l.status === 'ACTIVE').length;
                            const unpaid = fines.filter((f) => f.memberId === m.id && f.status === 'UNPAID').reduce((a, b) => a + b.amount, 0);
                            return (
                              <tr key={m.id}>
                                <td>
                                  <div className="w-8 h-8 rounded-full bg-gradient-to-tr from-[#9C27B0] to-[#1565C0] flex items-center justify-center font-bold text-xs text-white">
                                    {m.name.slice(0, 2).toUpperCase()}
                                  </div>
                                </td>
                                <td className="font-semibold text-white">{m.name}</td>
                                <td className="font-mono text-xs text-[#CE93D8]">{m.membershipId}</td>
                                <td><span className="px-2.5 py-0.5 rounded-full text-[10px] font-semibold bg-white/10 text-[#CE93D8]">{m.memberType}</span></td>
                                <td><span className="badge-active">{active} / {m.maxBooks || 3} books</span></td>
                                <td>{unpaid > 0 ? <span className="badge-overdue">â‚¹{unpaid}</span> : <span className="badge-active">Clear</span>}</td>
                              </tr>
                            );
                          })}
                        </tbody>
                      </table>
                    </div>
                  </div>
                )}

                {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                    PAGE 7: RESERVATIONS
                    â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
                {activeTab === 'reservations' && (
                  <div className="glass-card p-6 rounded-2xl space-y-5">
                    <div className="flex items-center justify-between">
                      <div>
                        <h2 className="font-heading text-lg font-bold text-gradient">Book Reservations Queue</h2>
                        <p className="text-xs text-[#7B6F8A]">Pending holds for checked-out library materials</p>
                      </div>
                    </div>

                    <div className="overflow-x-auto rounded-xl">
                      {sortedReservations.length === 0 ? (
                        <div className="p-12 text-center text-[#7B6F8A]"><span className="text-4xl block mb-2">ðŸ”–</span>No reservations on file.</div>
                      ) : (
                        <table>
                          <thead>
                            <tr>
                              <th onClick={() => sortReservations('id')}>Res ID {getResSortIcon('id')}</th>
                              <th onClick={() => sortReservations('memberName')}>Member {getResSortIcon('memberName')}</th>
                              <th onClick={() => sortReservations('bookTitle')}>Book Title {getResSortIcon('bookTitle')}</th>
                              <th onClick={() => sortReservations('date')}>Date Placed {getResSortIcon('date')}</th>
                              <th>Status</th>
                              <th className="text-right">Action</th>
                            </tr>
                          </thead>
                          <tbody>
                            {sortedReservations.map((r) => (
                              <tr key={r.id}>
                                <td className="font-mono text-xs text-[#CE93D8]">{r.id}</td>
                                <td className="font-semibold text-white">{r.memberName}</td>
                                <td className="text-white truncate max-w-xs">{r.bookTitle}</td>
                                <td className="text-xs text-[#7B6F8A]">{r.date}</td>
                                <td>{r.status === 'PENDING' ? <span className="badge-pending">PENDING</span> : <span className="badge-returned">CANCELLED</span>}</td>
                                <td className="text-right">
                                  {r.status === 'PENDING' && (
                                    <button onClick={() => handleCancelReservation(r.id)} className="btn-danger px-2.5 py-1 text-xs">Cancel</button>
                                  )}
                                </td>
                              </tr>
                            ))}
                          </tbody>
                        </table>
                      )}
                    </div>
                  </div>
                )}

                {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
                    PAGE 8: REPORTS & AUDIT
                    â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
                {activeTab === 'reports' && (
                  <div className="space-y-6">
                    <div className="flex items-center justify-between no-print">
                      <div>
                        <h2 className="font-heading text-lg font-bold text-gradient">System Reports & Audit</h2>
                        <p className="text-xs text-[#7B6F8A]">Print-ready inventory, overdue audit, and accounts breakdown</p>
                      </div>
                      <button onClick={() => window.print()} className="btn-primary py-2 px-4 text-xs font-bold flex items-center gap-2">
                        <span>ðŸ–¨ï¸</span> Print Report
                      </button>
                    </div>

                    <div className="glass-card p-6 rounded-2xl space-y-4">
                      <h3 className="font-heading text-sm font-bold text-gradient-purple">1. Complete Catalogue Status</h3>
                      <div className="overflow-x-auto">
                        <table>
                          <thead>
                            <tr>
                              <th>ISBN</th><th>Title</th><th>Author</th><th>Category</th><th>Total Copies</th><th>Available</th>
                            </tr>
                          </thead>
                          <tbody>
                            {books.map((b) => (
                              <tr key={b.isbn}>
                                <td className="font-mono text-xs text-[#CE93D8]">{b.isbn}</td>
                                <td className="font-semibold text-white">{b.title}</td>
                                <td>{b.author}</td><td>{b.category}</td>
                                <td className="text-center font-bold">{b.totalCopies}</td>
                                <td className="text-center">{b.availableCopies}</td>
                              </tr>
                            ))}
                          </tbody>
                        </table>
                      </div>
                    </div>

                    <div className="glass-card p-6 rounded-2xl space-y-4">
                      <h3 className="font-heading text-sm font-bold text-[#FF5252]">2. Overdue Loans Audit</h3>
                      <div className="overflow-x-auto">
                        {loans.filter((l) => l.status === 'ACTIVE' && l.dueDate < getTodayStr()).length === 0 ? (
                          <p className="text-xs text-[#69F0AE] py-3">No loans are currently overdue.</p>
                        ) : (
                          <table>
                            <thead>
                              <tr>
                                <th>Loan ID</th><th>Member</th><th>Book</th><th>Due Date</th><th>Overdue Days</th><th>Estimated Fine</th>
                              </tr>
                            </thead>
                            <tbody>
                              {loans.filter((l) => l.status === 'ACTIVE' && l.dueDate < getTodayStr()).map((l) => {
                                const days = getDiffDays(l.dueDate);
                                return (
                                  <tr key={l.id} className="bg-red-500/[0.08]">
                                    <td className="font-mono text-xs text-[#CE93D8]">{l.id}</td>
                                    <td className="font-semibold text-white">{l.memberName}</td>
                                    <td>{l.bookTitle}</td>
                                    <td className="text-[#FF5252] font-semibold">{l.dueDate}</td>
                                    <td>{days} days</td>
                                    <td className="font-bold text-[#FF5252]">â‚¹{days * settings.fineRate}</td>
                                  </tr>
                                );
                              })}
                            </tbody>
                          </table>
                        )}
                      </div>
                    </div>

                    <div className="glass-card p-6 rounded-2xl space-y-4">
                      <h3 className="font-heading text-sm font-bold text-[#FFD740]">3. Fine Accounts Revenue</h3>
                      <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                        <div className="p-4 rounded-xl bg-white/[0.03] border border-white/5">
                          <span className="text-xs text-[#7B6F8A] block mb-1">Settled Fines</span>
                          <span className="font-heading text-3xl font-extrabold text-[#69F0AE]">
                            â‚¹{fines.filter((f) => f.status === 'SETTLED').reduce((a, b) => a + b.amount, 0)}
                          </span>
                        </div>
                        <div className="p-4 rounded-xl bg-white/[0.03] border border-white/5">
                          <span className="text-xs text-[#7B6F8A] block mb-1">Pending Unpaid Dues</span>
                          <span className="font-heading text-3xl font-extrabold text-[#FF5252]">
                            â‚¹{fines.filter((f) => f.status === 'UNPAID').reduce((a, b) => a + b.amount, 0)}
                          </span>
                        </div>
                      </div>
                    </div>
                </div>
              )}
            </main>
          </div>
        </div>
      )}

      {/* â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€
          MODALS
          â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€ */}
      {/* Book Modal */}
      <Modal isOpen={bookModalOpen} onClose={() => setBookModalOpen(false)} title={editingBook ? 'Edit Book Details' : 'Register New Book'}>
        <form onSubmit={handleSaveBook} className="space-y-4">
          <FloatingInput label="ISBN Number" value={bookForm.isbn} onChange={(e) => setBookForm({ ...bookForm, isbn: e.target.value })} disabled={!!editingBook} error={bookErrors.isbn} required />
          <FloatingInput label="Book Title" value={bookForm.title} onChange={(e) => setBookForm({ ...bookForm, title: e.target.value })} error={bookErrors.title} required />
          <FloatingInput label="Author" value={bookForm.author} onChange={(e) => setBookForm({ ...bookForm, author: e.target.value })} error={bookErrors.author} required />
          <FloatingInput label="Category" value={bookForm.category} onChange={(e) => setBookForm({ ...bookForm, category: e.target.value })} error={bookErrors.category} required />
          <FloatingInput label="Total Copies" type="number" min="1" value={bookForm.copies} onChange={(e) => setBookForm({ ...bookForm, copies: e.target.value })} error={bookErrors.copies} required />
          <button type="submit" className="btn-primary w-full py-3 text-xs font-bold uppercase mt-2">{editingBook ? 'Update Book' : 'Add to Inventory'}</button>
        </form>
      </Modal>

      {/* Return Modal */}
      <Modal isOpen={returnModalOpen} onClose={() => setReturnModalOpen(false)} title="Confirm Book Return">
        {selectedLoanReturn && (() => {
          const overdue = getDiffDays(selectedLoanReturn.dueDate);
          const fine = overdue > 0 ? overdue * settings.fineRate : 0;
          return (
            <div className="space-y-4">
              <div className="p-4 rounded-xl bg-white/[0.04] border border-white/10 space-y-1">
                <p className="text-sm font-bold text-white">{selectedLoanReturn.bookTitle}</p>
                <p className="text-xs text-[#CE93D8]">Patron: {selectedLoanReturn.memberName} ({selectedLoanReturn.membershipId})</p>
                <p className="text-xs text-[#7B6F8A] pt-1">Due Date: {selectedLoanReturn.dueDate}</p>
              </div>
              {fine > 0 ? (
                <div className="p-4 rounded-xl bg-red-500/[0.1] border border-red-500/20 text-center">
                  <span className="text-xs text-[#FF5252] font-bold block mb-1">OVERDUE BY {overdue} DAYS</span>
                  <span className="font-heading text-3xl font-extrabold text-[#FF5252]">Fine: â‚¹{fine}</span>
                </div>
              ) : (
                <div className="p-4 rounded-xl bg-green-500/[0.1] border border-green-500/20 text-center text-[#69F0AE] font-bold text-xs">
                  Returned On Time â€” Zero Fine!
                </div>
              )}
              <div className="flex gap-2.5 pt-2">
                {fine > 0 ? (
                  <>
                    <button onClick={() => handleConfirmReturn(false)} className="btn-secondary flex-1 py-3 text-xs font-bold">Defer Fine</button>
                    <button onClick={() => handleConfirmReturn(true)} className="btn-primary flex-1 py-3 text-xs font-bold">Pay â‚¹{fine} & Return</button>
                  </>
                ) : (
                  <button onClick={() => handleConfirmReturn(false)} className="btn-primary w-full py-3 text-xs font-bold">Confirm Return</button>
                )}
              </div>
            </div>
          );
        })()}
      </Modal>

      {/* Member Modal */}
      <Modal isOpen={memberModalOpen} onClose={() => setMemberModalOpen(false)} title="Register New Patron">
        <form onSubmit={handleRegisterMember} className="space-y-4">
          <FloatingInput label="Full Name" value={memberForm.name} onChange={(e) => setMemberForm({ ...memberForm, name: e.target.value })} error={memberErrors.name} required />
          <FloatingInput label="Username" value={memberForm.username} onChange={(e) => setMemberForm({ ...memberForm, username: e.target.value })} error={memberErrors.username} required />
          <FloatingInput label="Password" type="password" value={memberForm.password} onChange={(e) => setMemberForm({ ...memberForm, password: e.target.value })} error={memberErrors.password} required />
          <FloatingInput label="Email" type="email" value={memberForm.email} onChange={(e) => setMemberForm({ ...memberForm, email: e.target.value })} error={memberErrors.email} required />
          <div>
            <label className="block text-xs font-semibold text-[#CE93D8] mb-1">Membership Quota *</label>
            <select value={memberForm.memberType} onChange={(e) => setMemberForm({ ...memberForm, memberType: e.target.value })} className="glass-select">
              <option value="Student">Student (Max 3 Books)</option>
              <option value="Faculty">Faculty (Max 5 Books)</option>
            </select>
          </div>
          <button type="submit" className="btn-primary w-full py-3 text-xs font-bold uppercase mt-2">Register Member</button>
        </form>
      </Modal>
    </div>
  );
}

ReactDOM.createRoot(document.getElementById('root')).render(<App />);
</script>
</body>
</html>

```